App: PiramidGame IFPR-Pgua

IFPR – Campus Paranaguá

1. Ideia geral

O aplicativo **PiramidGame IFPR-Pgua** nasceu em meio a uma crise sem precedentes no **IFPR – Campus Paranaguá**: Coquinha percebeu que o campus andava meio anêmico, sem brilho, sem resenha e perigosamente próximo da normalidade.

Como este é seu último ano, Coquinha decidiu que não poderia simplesmente passar pelo IFPR sem deixar uma marca histórica. Entre uma conversa e outra, discutiu com Yumi a criação de um aplicativo capaz de ranquear os alunos por popularidade, carisma, presença, caos e outros talentos socialmente questionáveis, porém altamente valorizados nos corredores do campus.

Antes que a ideia virasse apenas mais uma conversa perdida no intervalo, os dois resolveram consultar quem poderia dizer se aquilo era tecnicamente possível. Foi aí que entraram em cena Matheus Grêmio, da INFO23, e Matheus, da INFO24, avaliando a viabilidade do projeto e tentando descobrir se dava para transformar fofoca institucional em código Flutter.

Só que o projeto cresceu.

Guardar tudo apenas no celular já não era suficiente. Se a intenção era descobrir quem realmente dominava o **Nível Lenda**, seria necessário colocar todo mundo no mesmo universo. Nada de um ranking diferente em cada aparelho.

A solução encontrada foi levar a confusão para a nuvem utilizando os serviços do **Firebase**, com autenticação dos usuários e armazenamento dos dados no **Firestore**.

Sofia, que estava por perto ouvindo tudo, se empolgou com a proposta e se ofereceu para ajudar. Logo, Ana Júlia e Bárbara também entraram na missão de definir os critérios que separariam os meros estudantes dos verdadeiros populares do campus.

A notícia se espalhou rapidamente. Em pouco tempo, Pedro Rodrigo e Lussani já estavam envolvidos com parte da especificação do projeto, tentando organizar o caos antes que ele virasse requisito obrigatório sem ninguém perceber.

Mas toda grande ideia precisa de uma opinião de peso. Foi então que Monão entrou na história. Não ajudou exatamente em nada, mas deu palpite, porque em projeto de aluno sempre existe alguém que não implementa, não documenta, mas aparece para sugerir mudança na reta final.

Ioshi, da INFO26, também tentou dar sua contribuição, mas acabou levando um raspa de Mounir. Afinal, segundo a ordem natural do campus, calouro até pode ter opinião, mas precisa esperar sua hora para dar palpite com propriedade.

Enquanto isso, Bizzon, preocupado com a possibilidade real de ser menos popular que Topete, resolveu agir nos bastidores e mandou sugestões pela boca de sacola da Rúbia. A situação ficou ainda mais séria quando Luizinho percebeu uma oportunidade única de mudar sua imagem pública e já começou sua campanha informal rumo ao topo do ranking.

Diante desse cenário, a sorte está lançada.

Agora caberá aos alunos de **TADS** resolverem esse B.O. e implementarem o aplicativo a partir das sugestões dessa turma toda.

O resultado será um sistema desenvolvido em **Flutter**, com autenticação de usuários por meio do **Firebase Authentication**, armazenamento remoto utilizando o **Firestore** e possibilidade de avaliar pessoas do IFPR – Campus Paranaguá segundo critérios descontraídos de popularidade.

Cada usuário autenticado terá sua própria visão das avaliações realizadas, mas o sistema também deverá ser capaz de consolidar as avaliações dos diferentes usuários para produzir o tão aguardado **ranking global do campus**.

No fim, o **PiramidGame IFPR-Pgua** não será apenas um app.

Será um registro histórico da convivência, da resenha, do caos controlado e da luta eterna por um lugar no topo do **Nível Lenda**.

---

2. Pré-requisitos técnicos do projeto

O aplicativo deverá obrigatoriamente:

- utilizar **Flutter** para o desenvolvimento;
- utilizar **Firebase Authentication** para autenticação dos usuários;
- utilizar **Firestore** como banco de dados remoto;
- permitir autenticação utilizando **e-mail e senha**;
- permitir autenticação utilizando uma **conta Google**;
- utilizar **SharedPreferences** somente para armazenamento local de preferências, cache e informações auxiliares relacionadas à sessão;
- utilizar o pacote **go_router** para navegação entre telas;
- utilizar **Flutter Signals** para gerenciamento de estado e reatividade;

- utilizar um arquivo próprio de **tema** para definir a identidade visual da aplicação;
- permitir alternância entre **tema claro e escuro** em tempo de execução;
- utilizar o padrão **MVVM**;
- utilizar o padrão **Command** para disparar ações da interface;
- utilizar o padrão **Result** para encapsular retornos de sucesso e erro entre as camadas;
- programar respeitando os princípios do **SOLID**;
- organizar o projeto segundo os princípios da **Arquitetura Limpa**;
- respeitar a organização arquitetural utilizada na disciplina de Dispositivos Móveis.

O **Firestore será obrigatório como banco de dados remoto da aplicação**.

Não será permitido utilizar SharedPreferences, SQLite, Hive ou arquivos JSON como mecanismo principal de persistência dos dados.

---

3. Serviços obrigatórios

3.1 Firebase Authentication

O **Firebase Authentication** será responsável pela autenticação dos usuários da aplicação.

O aplicativo deverá permitir pelo menos duas formas de autenticação.

E-mail e senha

O usuário poderá criar uma conta informando:

- e-mail;
- senha.

Ao realizar o cadastro, deverá ser criado um usuário no **Firebase Authentication**.

Posteriormente, o usuário poderá acessar novamente a aplicação utilizando essas credenciais.

Conta Google

O usuário também deverá possuir a opção de autenticação utilizando sua **conta Google**.

Nesse caso, a autenticação deverá ser integrada ao Firebase Authentication.

3.2 Firestore

O **Firestore** será o banco de dados remoto da aplicação.

Deverão ser armazenados no Firestore, entre outros:

- usuários;
- pessoas disponíveis para avaliação;
- avaliações realizadas;
- notas dos critérios;
- informações necessárias para a geração dos rankings.

Como os dados estarão armazenados remotamente, o usuário deverá conseguir acessar suas informações mesmo utilizando outro dispositivo, desde que esteja autenticado com a mesma conta.

O Firestore deverá ser acessado exclusivamente pelas camadas responsáveis pelo acesso aos dados.

A interface gráfica não deverá acessar o banco diretamente.

3.3 SharedPreferences

O **SharedPreferences não será utilizado como banco de dados principal da aplicação**.

Sua utilização ficará restrita a informações locais simples, tais como:

- preferência entre tema claro e escuro;
- informações auxiliares da sessão;
- cache simples relacionado ao usuário autenticado;
- configurações locais da aplicação.

O SharedPreferences poderá auxiliar na experiência de sessão do usuário, evitando procedimentos desnecessários ao abrir novamente o aplicativo.

Entretanto, a autenticação válida deverá continuar sendo determinada pelo **Firebase Authentication**.

A senha do usuário **não deverá ser armazenada no SharedPreferences**.

---

4. Organização arquitetural obrigatória

O projeto deverá seguir a seguinte organização em camadas:

```
services → repositories → use cases → facade de use cases → viewmodel → UI
```

4.1 Services

A camada **services** será responsável pelo acesso direto aos recursos externos e locais utilizados pela aplicação.

Neste projeto poderão existir, por exemplo:

- `AuthService`;
- `FirestoreService`;
- `LocalStorageService`;
- `GoogleAuthService`.

O `AuthService` será responsável pela comunicação com o **Firebase Authentication**.

O `FirestoreService` será responsável pela comunicação com o **Firestore**.

O `LocalStorageService` poderá utilizar **SharedPreferences**.

A camada de UI não deverá conhecer diretamente esses serviços.

4.2 Repositories

A camada **repositories** será responsável por intermediar o acesso aos dados.

Poderão existir, por exemplo:

- `AuthRepository`;
- `PessoaRepository`;
- `AvaliacaoRepository`;
- `RankingRepository`;
- `PreferenceRepository`.

Os repositories deverão utilizar os services para acessar os recursos necessários.

Um repository não deverá depender diretamente da interface gráfica.

4.3 Use Cases

A camada de **Use Cases** será responsável pelas regras de negócio da aplicação.

Exemplos de use cases:

- cadastrar usuário;
- autenticar usuário com e-mail e senha;
- autenticar usuário com Google;
- verificar usuário autenticado;
- realizar logout;
- cadastrar pessoa;
- alterar pessoa;
- remover pessoa;
- buscar todas as pessoas;
- buscar pessoa por identificador;
- registrar avaliação;
- alterar avaliação;
- consultar avaliação;
- buscar avaliações realizadas pelo usuário autenticado;
- calcular ranking pessoal;
- calcular ranking global;
- verificar tentativa de autoavaliação;
- alterar tema do aplicativo.

4.4 Facade de Use Cases

A facade deverá agrupar os use cases relacionados, evitando que a ViewModel precise conhecer muitos casos de uso individualmente.

Exemplos:

AuthFacade

Poderá disponibilizar:

- criar conta;
- login;
- login com Google;
- verificar autenticação;
- logout.

PessoaFacade

Poderá disponibilizar:

- cadastrar;
- buscar;
- listar;
- alterar;
- remover.

AvaliacaoFacade

Poderá disponibilizar:

- avaliar;
- alterar avaliação;
- consultar avaliação;
- carregar avaliações.

RankingFacade

Poderá disponibilizar:

- gerar ranking pessoal;
- gerar ranking global.

4.5 ViewModel

A **ViewModel** será responsável por controlar o estado da interface e intermediar a comunicação entre a UI e os casos de uso.

A ViewModel deverá utilizar **Flutter Signals** para manter os dados reativos.

Também deverá utilizar **Commands** para disparar operações.

Exemplos:

- `loginCommand`;
- `loginGoogleCommand`;
- `logoutCommand`;
- `cadastrarPessoaCommand`;
- `alterarPessoaCommand`;
- `removerPessoaCommand`;
- `avaliarPessoaCommand`;
- `carregarRankingPessoalCommand`;

- `carregarRankingGlobalCommand`.

4.6 UI

A camada de **UI** será responsável exclusivamente pelas telas e componentes visuais do aplicativo.

A UI não deverá acessar diretamente:

- Firebase Authentication;
- Firestore;
- SharedPreferences;
- repositories;
- services.

A UI deverá se comunicar com a respectiva **ViewModel**.

---

5. Usuário autenticado e pessoa avaliada

Neste projeto deverá existir uma distinção importante entre:

usuário autenticado e **pessoa avaliada**.

O **usuário autenticado** é aquele que possui uma conta e realizou login na aplicação.

A **pessoa avaliada** é alguém cadastrado no sistema e que poderá receber avaliações.

Esses dois conceitos não deverão ser tratados obrigatoriamente como o mesmo registro.

Uma pessoa poderá ser cadastrada para receber avaliações sem possuir uma conta de autenticação.

Da mesma forma, um usuário autenticado poderá estar relacionado a uma pessoa existente no sistema.

Essa relação será importante para impedir a autoavaliação.

---

6. Autenticação

Antes de acessar as principais funcionalidades, o usuário deverá estar autenticado.

Ao iniciar o aplicativo:

1. deverá ser exibida a tela de Splash;
2. o aplicativo deverá verificar o estado da autenticação;
3. caso exista uma autenticação válida, o usuário deverá ser direcionado para a área principal da aplicação;
4. caso não exista uma autenticação válida, deverá ser direcionado para a tela de login.

O sistema deverá permitir:

- cadastro utilizando e-mail e senha;
- login utilizando e-mail e senha;
- login utilizando conta Google;
- logout.

---

7. Cadastro de pessoas

O usuário autenticado poderá cadastrar pessoas para serem avaliadas.

Cada pessoa deverá possuir os seguintes dados:

- nome;
- curso;
- turma/ano;
- apelido;
- data de nascimento.

Essas informações formarão o perfil básico da pessoa dentro do aplicativo.

O cadastro de uma pessoa no Firestore **não significa a criação automática de uma conta de autenticação**.

---

8. Pré-definições do cadastro

8.1 Cursos disponíveis

O campo **Curso** deverá ser selecionado entre:

- INFO;
- MEC;

- MAMB;
- PROD;
- TADS;
- TGA.

8.2 Turma/ano

O campo **Turma/ano** deverá permitir selecionar um ano entre:

1998 e 2026.

A utilização de seleção deverá evitar digitação manual desnecessária.

8.3 Campos de digitação e seleção

Campos com digitação:

- nome;
- apelido.

Campos com seleção ou componentes específicos:

- curso;
- turma/ano;
- data de nascimento;
- critérios de avaliação.

A data de nascimento deverá ser selecionada preferencialmente utilizando um componente como **DatePicker**.

---

9. Avaliação das pessoas

Uma das principais regras desta versão do projeto é que o **usuário autenticado deverá avaliar outras pessoas**.

O usuário não poderá realizar sua própria avaliação.

Portanto:

> **O usuário autenticado não poderá avaliar a si próprio.**

O usuário poderá cadastrar uma pessoa no sistema para que posteriormente ela possa ser avaliada.

Cada avaliação deverá identificar:

- usuário que realizou a avaliação;
- pessoa avaliada;
- notas atribuídas;
- pontuação total;
- data da avaliação.

Essas informações deverão ser armazenadas no **Firestore**.

---

10. Critérios de popularidade

Cada pessoa será avaliada utilizando **15 critérios**.

Cada critério deverá utilizar um sistema de **Star Rating**, permitindo notas de **1 a 5 estrelas**.

1. Resenha

Mede o quanto a pessoa anima a turma, puxa conversa e contribui para deixar o ambiente mais descontraído.

2. Presença VIP

Avalia o quanto a pessoa é lembrada, percebida ou reconhecida pelos colegas no dia a dia.

3. Aura

Representa a energia geral da pessoa: presença, estilo, jeito de ser e impacto que causa no ambiente.

4. Modo Parceiro

Mede o quanto ajuda os colegas, colabora nas atividades e demonstra espírito de parceria.

5. Carisma Natural

Avalia a facilidade para socializar, conversar e criar boas relações.

6. Humor de Milhões

Representa o quanto contribui com bom humor, brincadeiras saudáveis e momentos divertidos.

7. Energia de Grupo

Mede a participação em trabalhos, eventos, jogos, dinâmicas e atividades coletivas.

8. Criatividade Caótica

Avalia a capacidade de apresentar ideias diferentes, soluções inesperadas e comentários geniais.

9. Modo Atleta

Representa aptidão esportiva, disposição física e espírito competitivo saudável.

10. Talento de Palco

Mede aptidão artística, como música, canto, instrumentos, dança, ritmo ou presença em apresentações.

11. Drip Escolar

Avalia o estilo pessoal, considerando roupas, tênis, cabelo, acessórios e presença visual.

12. Coração de Dorama

Representa carisma afetivo, gentileza e aquela vibe de protagonista romântico, sem expor relacionamentos reais.

13. Queridinho dos Professores

Mede a boa relação com os professores, considerando respeito, participação, educação e responsabilidade.

14. Cérebro Turbo

Avalia o desempenho nos estudos, facilidade para aprender, resolver problemas e se destacar academicamente.

15. Caos Controlado

Mede o quanto a pessoa é bagunceira, zoeira ou imprevisível, mas ainda dentro dos limites do respeito e da convivência saudável.

---

11. Sistema de pontuação

Cada critério receberá uma nota de **1 a 5 estrelas**.

Como são 15 critérios, a pontuação máxima será:

15 × 5 = 75 pontos

A pontuação mínima será:

15 × 1 = 15 pontos

A soma das notas atribuídas formará o:

Nível Lenda

Exemplo:

Pessoa: João Pedro

- Resenha: 4
- Presença VIP: 3
- Aura: 5
- Modo Parceiro: 4
- Carisma Natural: 4
- Humor de Milhões: 5
- Energia de Grupo: 3
- Criatividade Caótica: 4
- Modo Atleta: 5
- Talento de Palco: 2
- Drip Escolar: 4
- Coração de Dorama: 3
- Queridinho dos Professores: 4
- Cérebro Turbo: 5
- Caos Controlado: 3

Nível Lenda: 58 pontos

---

12. Ranking pessoal

O aplicativo deverá possuir um **Ranking Pessoal**.

Esse ranking deverá considerar exclusivamente as avaliações realizadas pelo **usuário atualmente autenticado**.

Exemplo:

Um usuário realizou as seguintes avaliações:

- Ana Clara: 68 pontos;
- João Pedro: 58 pontos;
- Maria Eduarda: 54 pontos.

Seu ranking pessoal será:

1º Ana Clara – 68 pontos
2º João Pedro – 58 pontos
3º Maria Eduarda – 54 pontos

As avaliações realizadas pelos demais usuários não deverão interferir nesse ranking.

Portanto:

> **Ranking pessoal = ranking produzido exclusivamente a partir das avaliações feitas pelo usuário autenticado.**

---

13. Ranking global

O aplicativo também deverá possuir um **Ranking Global**.

Nesse ranking deverão ser consideradas as avaliações realizadas por **todos os usuários do aplicativo**.

Como uma mesma pessoa poderá receber avaliações de vários usuários, sua pontuação global deverá considerar a **média das avaliações recebidas**.

Exemplo:

Ana recebeu:

- avaliação 1: 60 pontos;
- avaliação 2: 70 pontos;
- avaliação 3: 65 pontos.

Sua pontuação global será:

(60 + 70 + 65) / 3 = 65 pontos

Assim:

Nível Lenda Global = 65 pontos

O ranking deverá ser ordenado da maior média para a menor.

Também deverá ser possível exibir a quantidade de avaliações recebidas.

Exemplo:

1º Ana Clara – 68,4 pontos – 12 avaliações
2º João Pedro – 64,8 pontos – 9 avaliações
3º Maria Eduarda – 61,2 pontos – 15 avaliações

Portanto:

> **Ranking global = ranking produzido a partir das avaliações realizadas por todos os usuários.**

---

14. Funcionalidades obrigatórias

O aplicativo deverá permitir:

14.1 Criar conta

Cadastrar um usuário utilizando e-mail e senha por meio do Firebase Authentication.

14.2 Realizar login

Permitir autenticação utilizando:

- e-mail e senha;
- conta Google.

14.3 Manter sessão

Ao abrir novamente o aplicativo, o usuário não deverá precisar informar suas credenciais novamente enquanto houver uma autenticação válida.

14.4 Logout

Permitir o encerramento da sessão.

14.5 Cadastrar pessoa

Permitir o cadastro de uma nova pessoa no Firestore.

14.6 Listar pessoas

Mostrar as pessoas cadastradas.

A listagem poderá exibir:

- nome;
- apelido;
- curso;
- turma/ano;
- Nível Lenda, quando aplicável.

14.7 Visualizar detalhes

Permitir a visualização das informações completas de uma pessoa.

14.8 Alterar pessoa

Permitir a edição dos dados cadastrais.

14.9 Remover pessoa

Permitir a exclusão de uma pessoa conforme as regras definidas pelo sistema.

A remoção deverá solicitar confirmação antes da exclusão definitiva.

14.10 Avaliar pessoa

Permitir que o usuário autenticado atribua notas de 1 a 5 estrelas nos 15 critérios.

14.11 Alterar avaliação

Permitir que o usuário altere posteriormente uma avaliação realizada por ele.

14.12 Impedir autoavaliação

O sistema deverá impedir que o usuário autenticado avalie a si próprio.

14.13 Exibir ranking pessoal

Gerar um ranking utilizando exclusivamente as avaliações do usuário autenticado.

14.14 Exibir ranking global

Gerar um ranking utilizando as avaliações de todos os usuários armazenadas no Firestore.

14.15 Alternar tema

Permitir a troca entre tema claro e tema escuro em tempo de execução.

---

15. Estrutura sugerida para o Firestore

Uma possível organização dos dados será:

```
usuarios
    uidUsuario
        nome
        email
        foto
        provider

pessoas
    idPessoa
        nome
        apelido
        curso
        turma
        dataNascimento
        criadoPor

avaliacoes
    idAvaliacao
        avaliadorUid
        pessoaId
        resenha
        presencaVip
        aura
        modoParceiro
        carismaNatural
        humorDeMilhoes
        energiaDeGrupo
        criatividadeCaotica
        modoAtleta
        talentoDePalco
        dripEscolar
        coracaoDeDorama
        queridinhoProfessores
        cerebroTurbo
        caosControlado
        pontuacaoTotal
        dataAvaliacao
```

Essa estrutura serve como referência.

Os grupos poderão propor outra organização no Firestore, desde que ela permita atender corretamente às regras e funcionalidades definidas nesta especificação.

---

16. Regras relacionadas às avaliações

Cada avaliação deverá identificar claramente:

- quem avaliou;
- quem foi avaliado.

O sistema deverá impedir:

- avaliação sem usuário autenticado;
- autoavaliação;
- notas inferiores a 1;
- notas superiores a 5;
- criação de avaliações inválidas.

Cada usuário deverá possuir apenas **uma avaliação ativa para cada pessoa avaliada**.

Assim:

```
usuário + pessoa avaliada = uma avaliação
```

Caso deseje modificar sua opinião, o usuário deverá **alterar sua avaliação existente**, em vez de criar uma nova avaliação da mesma pessoa.

Essa regra evita que um usuário consiga aumentar ou diminuir artificialmente a pontuação global de alguém realizando várias avaliações.

---

17. Telas do aplicativo

O aplicativo deverá possuir obrigatoriamente:

- tela de **Splash**;
- tela de **Login**;
- tela ou funcionalidade de **Cadastro de usuário**;
- opção de autenticação com **Google**;
- tela de **Sobre o App**;
- tela para **listar pessoas**;

- tela para **cadastrar pessoa**;
- tela para **visualizar detalhes**;
- interface para **avaliar pessoa**;
- tela de **Ranking Pessoal**;
- tela de **Ranking Global**.

A organização das demais telas fica a critério dos grupos.

Poderão ser utilizados, por exemplo:

- menu principal;
- navegação por abas;
- menu inferior;
- menu lateral;
- telas separadas;
- diferentes seções em uma mesma tela.

O importante é que todas as funcionalidades obrigatórias estejam acessíveis e que a navegação seja clara.

---

18. Tela de Splash

A tela de Splash deverá ser exibida ao iniciar o aplicativo.

Ela poderá conter:

- nome do app;
- referência ao IFPR – Campus Paranaguá;
- ícone ou elemento visual relacionado ao tema;
- pequena frase de apresentação.

Durante essa etapa, o aplicativo deverá verificar o estado da autenticação.

Fluxo esperado:

```
Splash
   ↓
Verificar autenticação
   ↓
Usuário autenticado?
   ↓
 SIM                NÃO
   ↓                  ↓
Aplicação           Login
```

---

19. Tela de Login

A tela de Login deverá disponibilizar:

- campo de e-mail;
- campo de senha;
- botão **Entrar**;
- opção **Criar conta**;
- opção **Entrar com Google**.

Erros de autenticação deverão ser tratados adequadamente e informados ao usuário.

---

20. Tela Sobre o App

O aplicativo deverá possuir uma tela explicando:

- objetivo do app;
- contexto do IFPR – Campus Paranaguá;
- funcionamento dos critérios;
- cálculo do Nível Lenda;
- funcionamento do ranking pessoal;
- funcionamento do ranking global;
- utilização do Firestore;
- utilização do Firebase Authentication;
- utilização de tema claro e escuro.

Texto sugerido:

O PiramidGame IFPR-Pgua é um aplicativo desenvolvido em Flutter para fins didáticos. Ele permite cadastrar pessoas do IFPR – Campus Paranaguá e avaliá-las em critérios descontraídos de convivência, destaque e participação.

Cada avaliação atribui notas de 1 a 5 estrelas em 15 categorias. A soma dessas notas forma o Nível Lenda.

Cada usuário possui um ranking pessoal, baseado exclusivamente nas avaliações que realizou. O aplicativo também possui um ranking global, calculado a partir das avaliações realizadas pelos diferentes usuários.

Os dados são armazenados remotamente no **Firestore** e o acesso ao aplicativo utiliza **Firebase Authentication**.

O aplicativo também permite alternar entre tema claro e tema escuro.

---

21. Armazenamento dos dados

Os dados principais da aplicação deverão ser armazenados remotamente no **Firestore**.

Entre esses dados estão:

- usuários;
- pessoas cadastradas;
- avaliações;
- notas;
- dados necessários para os rankings.

O fluxo arquitetural deverá respeitar:

```
Flutter
   ↓
UI
   ↓
ViewModel
   ↓
Use Cases
   ↓
Repositories
   ↓
Services
   ↓
Firestore
```

Para leitura dos dados:

```
Firestore
   ↓
Services
   ↓
Repositories
   ↓
Use Cases
   ↓
ViewModel
   ↓
Flutter Signals
```

```
   ↓
UI
```

A aplicação não deverá utilizar SharedPreferences como substituto do Firestore.

---

22. Armazenamento local e sessão

O **SharedPreferences** será utilizado apenas como armazenamento auxiliar.

Por exemplo:

```
SharedPreferences
    ├── preferência de tema
    ├── dados simples de sessão
    └── cache do usuário autenticado
```

O SharedPreferences não deverá armazenar:

- lista principal de pessoas;
- avaliações;
- ranking;
- senha do usuário.

A autenticação deverá permanecer sob responsabilidade do **Firebase Authentication**.

---

23. Regras gerais do aplicativo

O aplicativo deverá respeitar as seguintes regras:

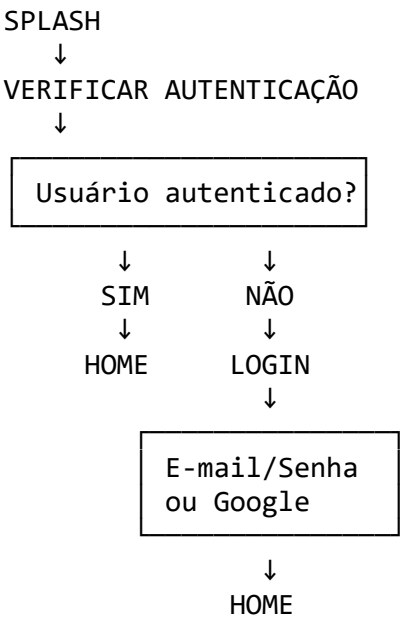
- somente usuários autenticados poderão acessar as funcionalidades principais;
- a autenticação deverá utilizar Firebase Authentication;
- deverá existir autenticação com e-mail e senha;
- deverá existir autenticação com conta Google;
- os dados principais deverão ser armazenados no Firestore;
- o SharedPreferences deverá ser utilizado apenas para informações locais simples;
- senhas não deverão ser armazenadas no SharedPreferences;
- o usuário não poderá avaliar a si próprio;
- pessoas poderão ser cadastradas para receber avaliações;

- cada avaliação deverá identificar o usuário avaliador;
- cada avaliação deverá identificar a pessoa avaliada;
- cada critério deverá possuir nota entre 1 e 5;
- a pontuação total deverá ser calculada automaticamente;
- cada usuário deverá possuir apenas uma avaliação ativa para cada pessoa;
- o ranking pessoal deverá utilizar apenas as avaliações do usuário autenticado;
- o ranking global deverá considerar as avaliações de todos os usuários;
- o ranking global deverá utilizar a média das avaliações recebidas;
- os dados deverão permanecer disponíveis após o fechamento da aplicação;
- a navegação deverá utilizar `go_router`;
- a gerência de estado deverá utilizar Flutter Signals;
- as ações deverão ser disparadas utilizando o padrão Command;
- os retornos entre camadas deverão utilizar o padrão Result;
- a arquitetura deverá seguir MVVM;
- deverão ser respeitados SOLID e Arquitetura Limpa;
- o aplicativo deverá permitir alternância entre tema claro e escuro.

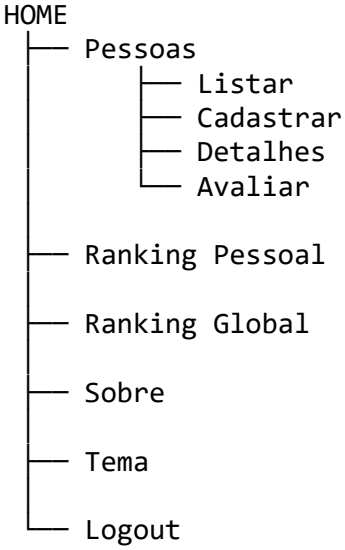
---

24. Fluxo básico da aplicação

Um fluxo possível será:



Na área principal:



---

25. Diferença entre os rankings

A diferença entre os dois rankings deverá estar clara na aplicação.

Ranking Pessoal

Responde à pergunta:

“Na minha opinião, quem possui o maior Nível Lenda?”

Considera exclusivamente as avaliações realizadas pelo usuário autenticado.

Ranking Global

Responde à pergunta:

“Na opinião dos usuários do aplicativo, quem possui o maior Nível Lenda?”

Considera as avaliações armazenadas no Firestore pelos diferentes usuários.

---

26. Resumo do projeto

O **PiramidGame IFPR-Pgua – Ranking de Popularidade** será uma aplicação Flutter com autenticação de usuários e armazenamento remoto.

O usuário deverá inicialmente realizar autenticação utilizando:

- e-mail e senha; ou

- conta Google.

A autenticação será realizada utilizando **Firebase Authentication**.

Depois de autenticado, o usuário poderá visualizar e cadastrar pessoas disponíveis para avaliação.

Cada pessoa poderá receber notas de **1 a 5 estrelas em 15 critérios**, produzindo uma pontuação entre **15 e 75 pontos**, denominada **Nível Lenda**.

O usuário autenticado não poderá avaliar a si próprio.

Cada usuário poderá possuir apenas uma avaliação ativa para cada pessoa avaliada, podendo posteriormente modificá-la.

As avaliações deverão ser armazenadas remotamente no **Firestore** e deverão estar relacionadas tanto ao usuário que realizou a avaliação quanto à pessoa avaliada.

O sistema possuirá dois rankings:

Ranking Pessoal: considera exclusivamente as avaliações realizadas pelo usuário autenticado.

Ranking Global: considera as avaliações realizadas pelos diferentes usuários e utiliza a média das avaliações recebidas por cada pessoa.

O **SharedPreferences** será utilizado somente para armazenamento local auxiliar, como preferências, cache e informações simples relacionadas à sessão.

O projeto utilizará:

- **Flutter**;
- **Firebase Authentication**;
- **Firestore**;
- **SharedPreferences**;
- **go_router**;
- **Flutter Signals**;
- **MVVM**;
- **Command**;
- **Result**;
- **SOLID**;
- **Arquitetura Limpa**.

Com isso, o projeto deixa de ser apenas um cadastro local com ranking e passa a funcionar como uma aplicação remota e multiusuário, capaz de descobrir não apenas quem é lenda na opinião de cada usuário, mas quem consegue sobreviver ao julgamento coletivo do **IFPR – Campus Paranaguá**.